

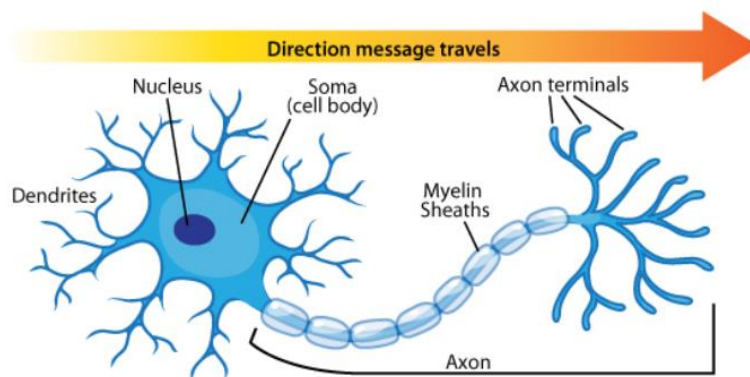
LAB 4

I. Neural network

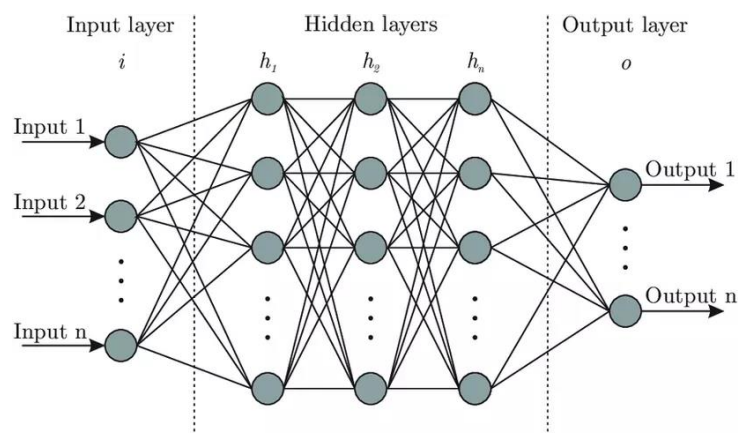
1. Cấu trúc của mạng Neural

Mạng neural nhân tạo (**Artificial Neural Network - ANN**) được lấy cảm hứng từ **neuron sinh học** trong não người:

- **Dendrites (sợi nhánh):** nhận tín hiệu từ neuron khác
- **Cell body (thân tế bào):** xử lý thông tin
- **Axon:** truyền tín hiệu đi
- **Synapse:** nơi kết nối giữa các neuron



Một mạng neural gồm nhiều lớp (layers)



Input layer (Lớp đầu vào)

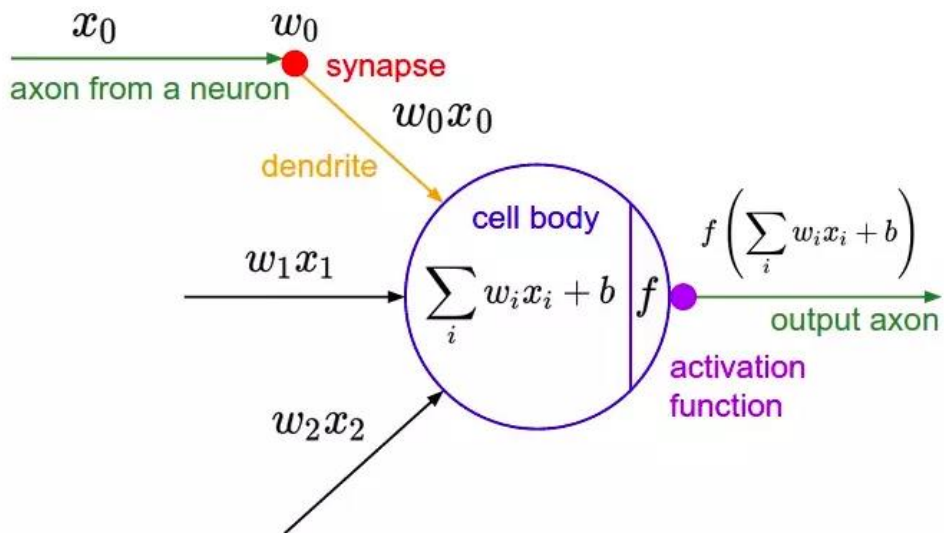
- Nhận dữ liệu ban đầu
- Ví dụ: ảnh, vector đặc trưng, tín hiệu

Hidden layers (Các lớp ẩn)

- Thực hiện xử lý và học đặc trưng
- Có thể có **1 hoặc nhiều lớp**
- Mỗi lớp gồm nhiều neuron

Output layer (Lớp đầu ra)

- Trả kết quả cuối cùng
- Ví dụ:
 - Phân lớp → xác suất
 - Hồi quy → giá trị số



Các thành phần

Input x_i

- Dữ liệu đầu vào
- Tương tự tín hiệu từ dendrites

Weight w_i

- Trọng số của mỗi input
- Quy định mức độ quan trọng của tín hiệu

Bias b

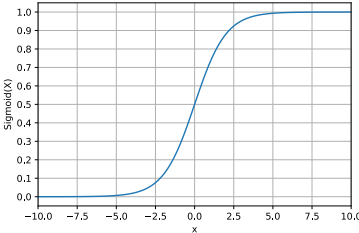
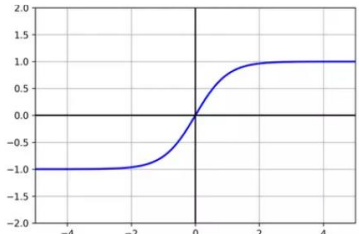
- Giá trị điều chỉnh
- Giúp mô hình linh hoạt hơn

Tổng có trọng số

$$\sum_i w_i x_i + b$$

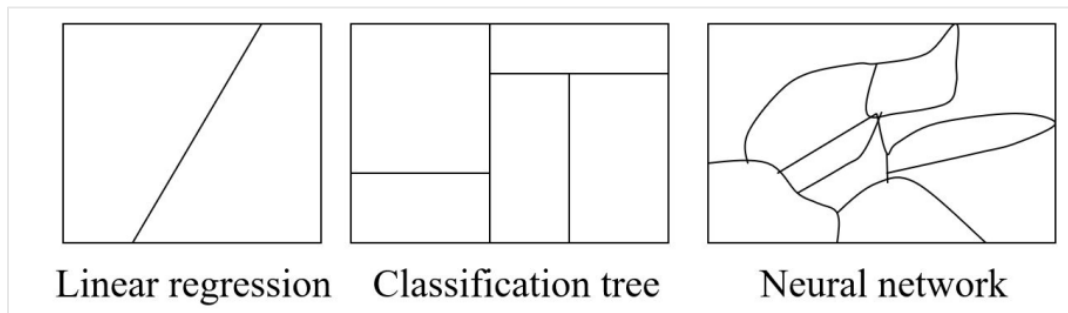
Activation function f

- Quyết định output có “kích hoạt” hay không
- Một số các hàm kích hoạt phổ biến:

Hàm Sigmoid	Hàm Tanh	Hàm Relu
$f(x) = \frac{1}{1 + e^{-x}}$	$f(x) = \frac{2}{1 + e^{-2z}} - 1$	$f(x) = \max(0, z)$
		
Hàm sẽ chuyển đổi các inputs thành các giá trị nằm trong khoảng $(0, 1)$	Các giá trị của hàm Tanh sẽ biến thiên trong khoảng $(-1, 1)$.	Giá trị đầu ra của hàm Relu nằm trong khoảng $[0, +\infty)$,

Nếu **không có activation**: toàn bộ mạng chỉ là tuyến tính (linear). Chính các Activation function giúp học các quan hệ phức tạp như hình ảnh hay ngôn ngữ, ...

Minh hoạ việc sử dụng các thuật toán khác nhau:



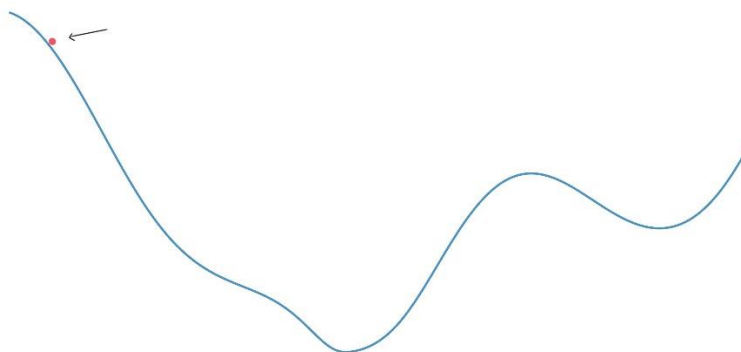
2. Thuật toán Gradient Descent

Thuật toán Gradient Descent là một phương pháp tối ưu hóa được sử dụng rộng rãi trong machine learning và các lĩnh vực liên quan. Thuật toán giúp tìm ra giá trị nhỏ nhất của một hàm số bằng cách điều chỉnh các tham số của hàm đó thông qua việc giảm gradient của hàm. Thuật toán thường được áp dụng cho các bài toán tối ưu, trong đó chúng ta cần tìm giá trị tối ưu của một hàm số có thể được đo bằng đạo hàm.

Dưới đây là cách thuật toán Gradient Descent hoạt động:

Giả sử ta có hàm số: $f(x)$

Bước 1: Khởi tạo ngẫu nhiên giá trị ban đầu cho các tham số của hàm số (khởi tạo ngẫu nhiên giá trị x), với giá trị khởi tạo ta sẽ có một giá trị $f(x)$ tương ứng với hàm số đã cho.



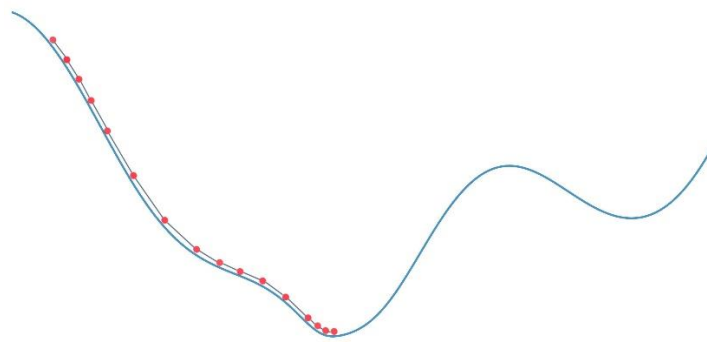
Đồ thị biểu diễn hàm $f(x)$

Bước 2: Lặp lại các bước sau cho đến khi đạt được điều kiện dừng:

- Tính toán gradient của hàm số tại vị trí hiện tại của các tham số. Gradient thể hiện hướng tăng nhanh nhất của hàm số tại một điểm cụ thể. $f'(x)$
- Di chuyển đến vị trí mới bằng cách cập nhật các tham số theo công thức:

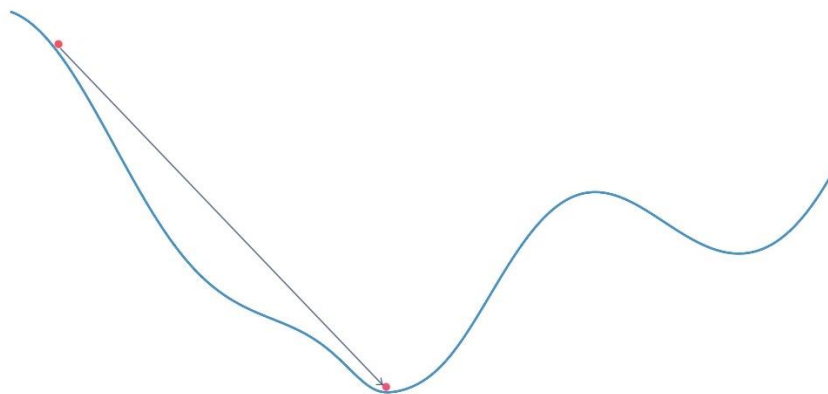
$$\text{Tham số mới} = \text{Tham số cũ} - \text{Learning_rate} * \text{Gradient}$$

Ở đây, learning_rate là một hằng số dương nhỏ được gọi là tỉ lệ học tập (learning rate), chỉ số này quyết định mức độ cập nhật của trọng số và tốc độ thay đổi của giá trị hàm số.



Các giá trị của hàm $f(x)$ ứng với các lần cập nhật

Bước 3: Trả về giá trị tối ưu của các tham số khi đạt được điều kiện dừng thì giá trị của hàm số tại lúc này là nhỏ nhất.



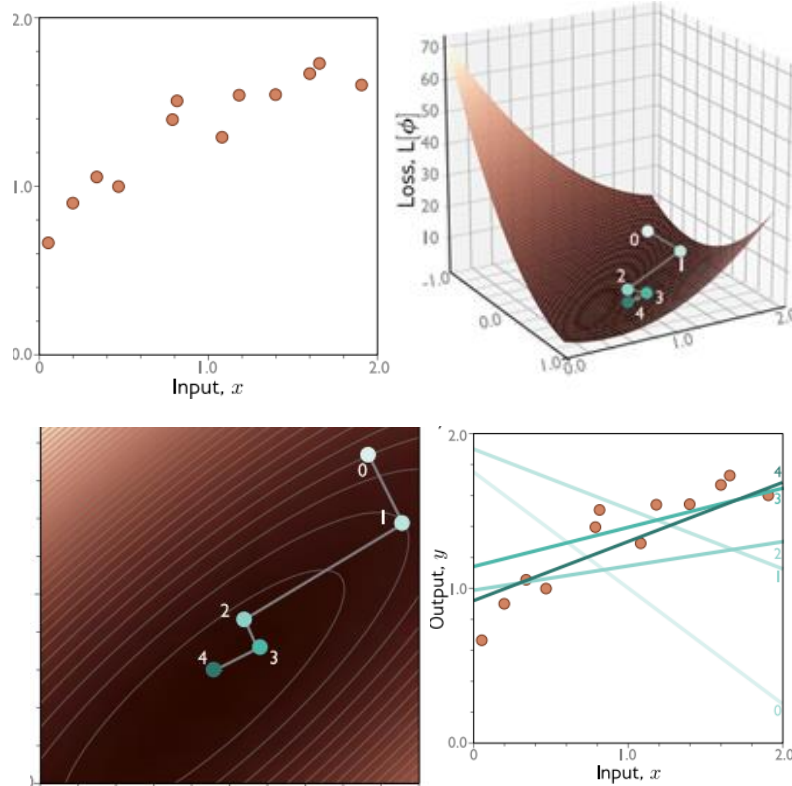
Vị trí của giá trị hàm $f(x)$ sau khi kết thúc thuật toán

Mã giả của thuật toán Gradient Descent:

Bước khởi tạo	Khởi tạo tham số
Bước 1	Tính đạo hàm của hàm loss theo các biến
Bước 2	Cập nhật các tham số mới
Bước 3	Lặp lại bước 1 và 2 đến khi thỏa điều kiện dừng

Mã giả của thuật toán Gradient Descent

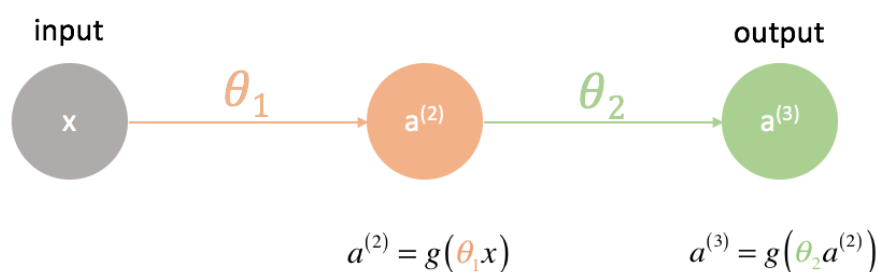
Hình hoạ cho quá trình cập nhật trọng số:



3. Cách mạng Neural cập nhật trọng số với Gradient Descent

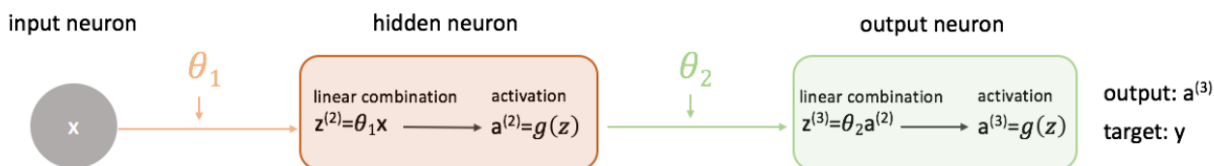
Các bước áp dụng để áp dụng Gradient Descent cho một mạng neural bao gồm:

Bước 1: Khởi tạo trọng số và bias: Trọng số và bias của mạng nơ-ron được khởi tạo ngẫu nhiên hoặc theo một phương pháp khác tùy thuộc vào thuật toán và kiến trúc mạng. Ở đây chúng ta có một mạng đơn giản với node đầu vào, một node ẩn và một node đầu ra. Mạng có các trọng số θ_1 và trọng số θ_2 đầu vào là x , hàm kích hoạt là $g(x)$.



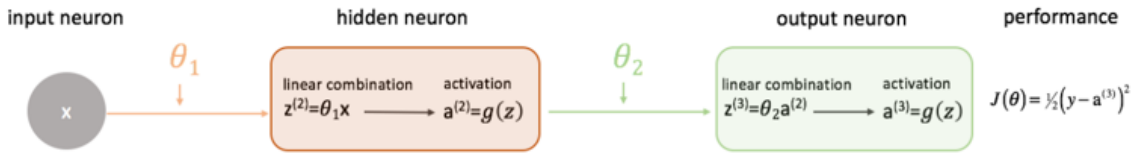
Quá trình lan truyền

Bước 2: Lan truyền tiến (forward propagation): Đưa dữ liệu huấn luyện qua mạng nơ-ron bằng quá trình lan truyền tiến để tính toán đầu ra của mạng cho mỗi mẫu dữ liệu. Công thức tính toán các giá trị cho các node và đầu ra như hình. Với input là x , output là $a^{(3)}$ và target là y .



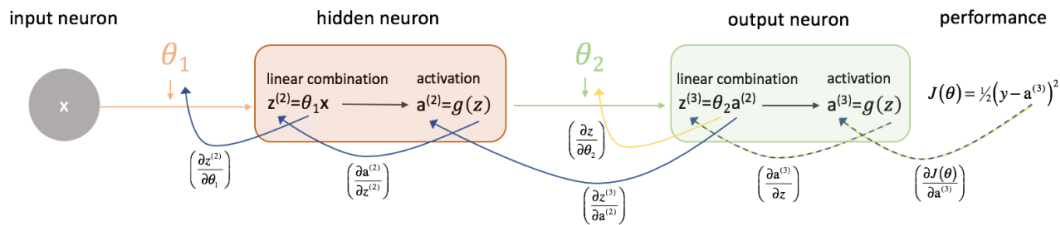
Tính toán lan truyền tiến

Bước 3: Tính toán độ lỗi (loss function): So sánh đầu ra dự đoán của mạng với giá trị đúng (nhãn) của dữ liệu huấn luyện để tính toán độ lỗi sử dụng một hàm mất mát (loss function). Ở đây chúng ta sẽ dùng hàm MSE làm hàm lỗi.



Tính toán độ lỗi

Bước 4: Lan truyền ngược (backpropagation): Tính toán gradient của hàm mất mát theo các trọng số và bias của mỗi nơ-ron trong mạng. Quá trình này được thực hiện bằng cách lan truyền ngược từ lớp cuối cùng đến lớp đầu vào.



Lan truyền ngược và cập nhật trọng số

Để giảm hàm lỗi cho mô hình chúng ta cần tính đạo hàm riêng đối với từng trọng số θ_1 và trọng số θ_2 của mô hình. Tương ứng với việc cần tính toán $\frac{\partial J(\theta)}{\partial \theta_1}$ và $\frac{\partial J(\theta)}{\partial \theta_2}$ (đạo hàm riêng của hàm mất mát theo từng trọng số θ_1 và trọng số θ_2)

- Với quy tắc Chain rule ta có:

$$\frac{\partial}{\partial z} p(q(z)) = \frac{\partial p}{\partial q} \frac{\partial q}{\partial z}$$

- Áp dụng Chain rule cho $\frac{\partial J(\theta)}{\partial \theta_2}$, ta được:

$$\frac{\partial J(\theta)}{\partial \theta_2} = \left(\frac{\partial J(\theta)}{\partial a^{(3)}} \right) \left(\frac{\partial a^{(3)}}{\partial z} \right) \left(\frac{\partial z}{\partial \theta_2} \right)$$

- Tương tự áp dụng Chain rule cho $\frac{\partial J(\theta)}{\partial \theta_1}$:

$$\frac{\partial J(\theta)}{\partial \theta_1} = \left(\frac{\partial J(\theta)}{\partial a^{(3)}} \right) \left(\frac{\partial a^{(3)}}{\partial z^{(3)}} \right) \left(\frac{\partial z^{(3)}}{\partial a^{(2)}} \right) \left(\frac{\partial a^{(2)}}{\partial z^{(2)}} \right) \left(\frac{\partial z^{(2)}}{\partial \theta_1} \right)$$

Bước 5: Cập nhật trọng số và bias: Sử dụng gradient tính được từ quá trình lan truyền ngược, cập nhật các trọng số và bias của mạng bằng phương pháp Gradient Descent. Công thức cập nhật thường được sử dụng là:

$$\theta_i := \theta_i + \Delta\theta_i$$

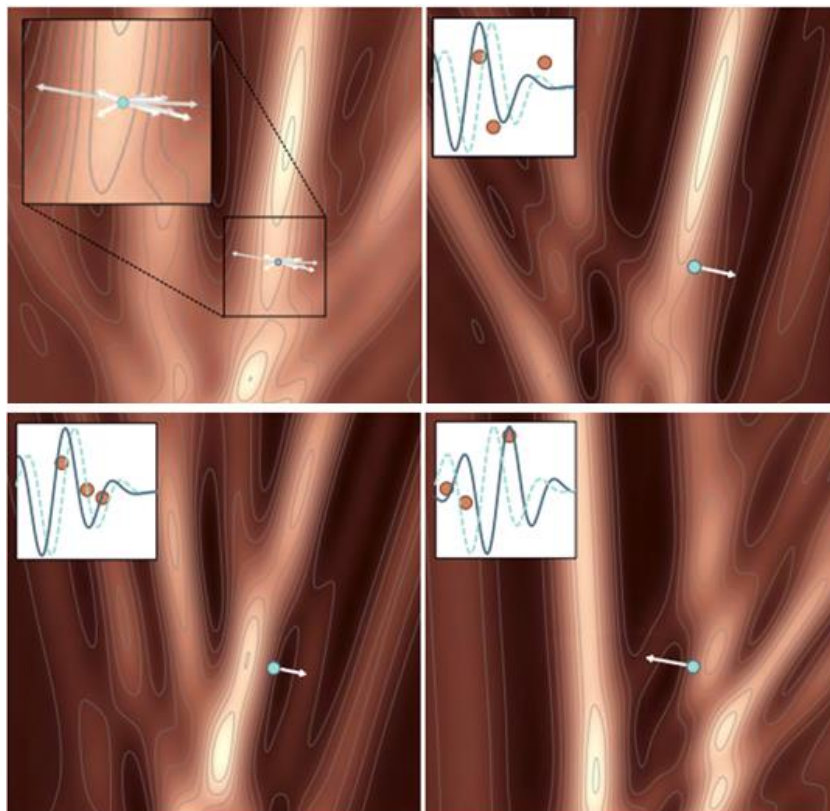
Trong đó, $\Delta\theta_i$ được tính bởi công thức:

$$\Delta\theta_i = -\eta \frac{\partial J(\theta)}{\partial \theta_i}$$

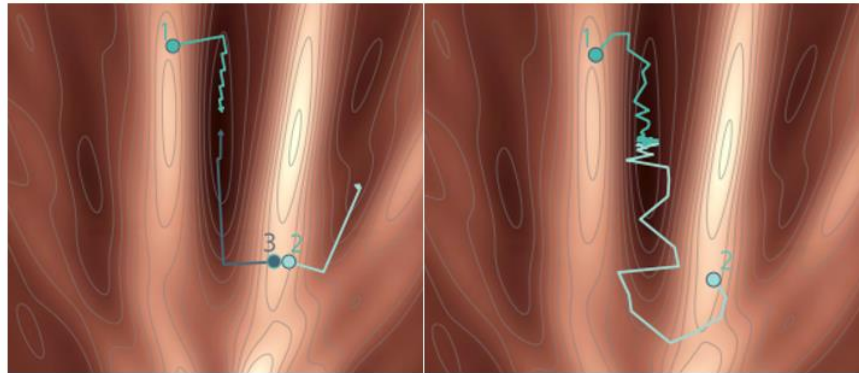
Với η là learning rate.

Với các mạng neural nhiều lớp khác chúng ta áp dụng các quy tắc tương tự để tính toán và cập nhật lại trọng số cho mạng.

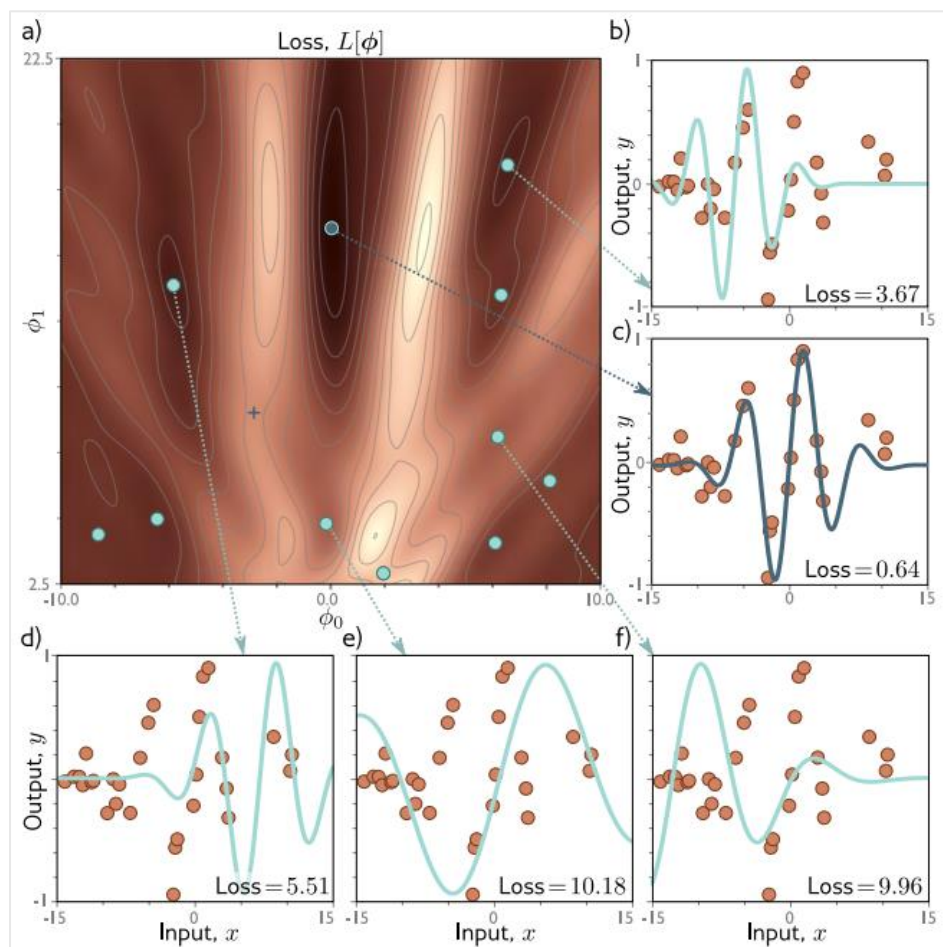
Với mỗi lần cập nhật, giá trị hàm loss thay đổi theo hướng của đạo hàm theo các trọng số



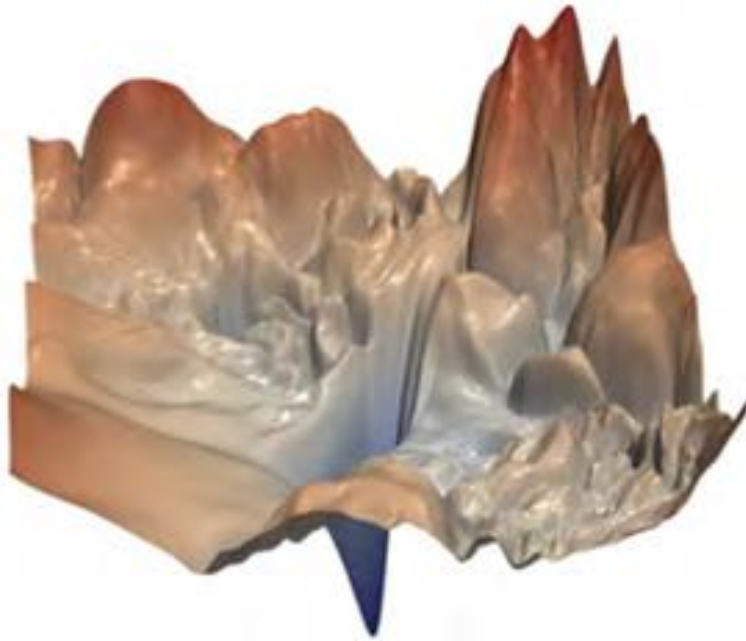
Các giá trị của hàm loss tiến về cực trị địa phương:



Ứng với mỗi cực trị ta có một giá trị hàm loss và độ khớp với dữ liệu cần biểu diễn (độ chính xác):

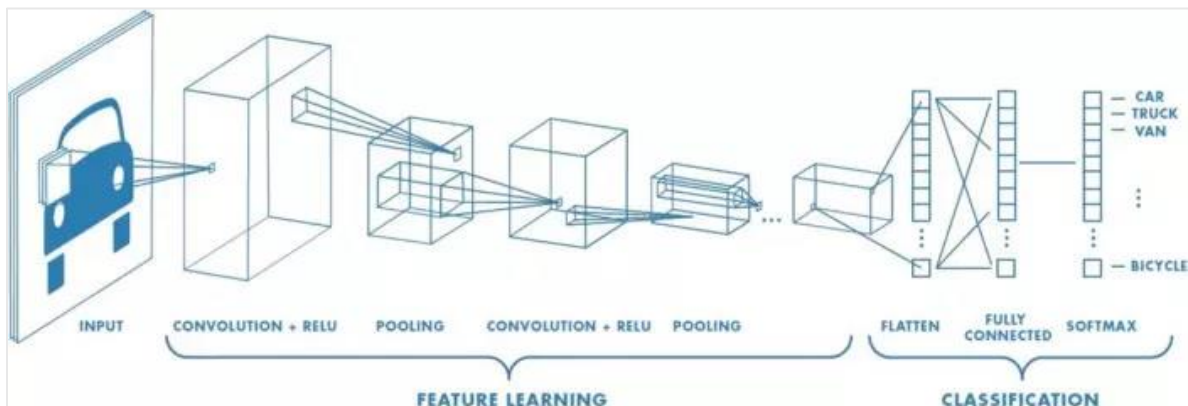


Minh hoạ hình dạng của hàm mất mát trong không gian:



II. Convolutional Neural networks

Mạng CNN là một mô hình học sâu chuyên dùng trong xử lý ảnh và thị giác máy tính. Mục tiêu của CNN là tự động học và trích xuất các đặc trưng từ dữ liệu ảnh, sau đó sử dụng các đặc trưng này để phân loại hoặc dự đoán.



Mạng neural tích chập

Lớp tích chập (Convolutional Layer)

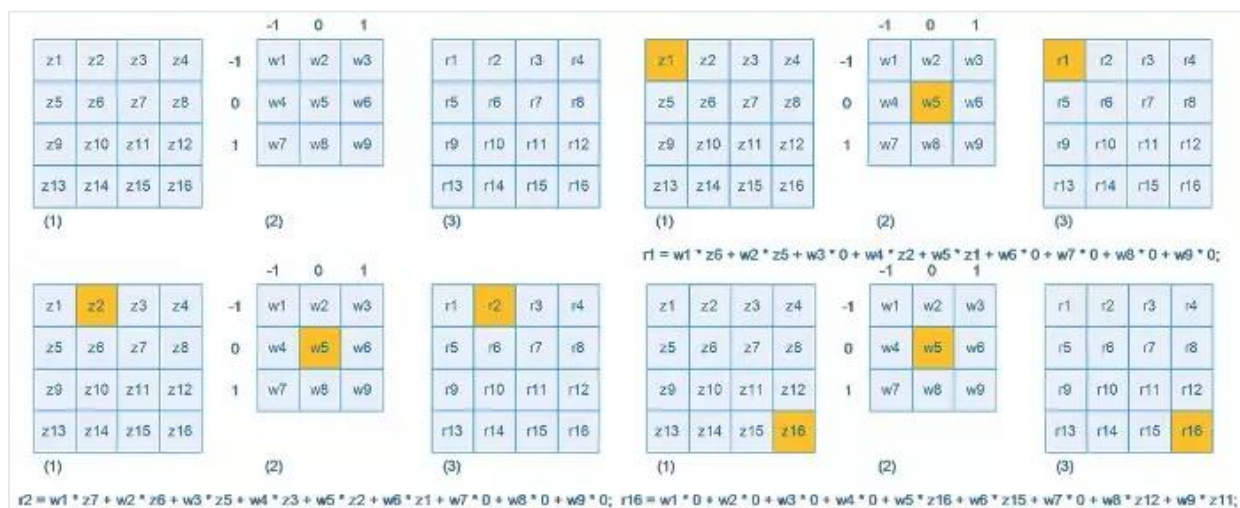
- Là thành phần quan trọng nhất của CNN.
- Thực hiện phép tích chập giữa ảnh đầu vào và kernel (bộ lọc).
- Mỗi kernel giúp phát hiện một loại đặc trưng (cạnh, góc, texture...).
- Kernel trượt trên toàn bộ ảnh để tạo ra **feature map**.

Phép toán tích chập được thực hiện bằng cách lấy một ma trận đầu vào (thường là một ma trận 2D) và áp dụng một ma trận bộ lọc (kernel) lên đó. Mỗi phần tử trong ma trận kết quả được tính bằng cách nhân các phần tử tương ứng trong ma trận đầu vào với các phần tử tương ứng trong ma trận bộ lọc, sau đó tính tổng các kết quả này. Quá trình này được lặp lại trên toàn bộ ma trận đầu vào bằng cách dịch chuyển ma trận bộ lọc qua đầu vào theo từng bước (stride).

image		kernel		result																																																											
<table><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr></table>	1	0	0	1	0	0	1	1	0	1	1	0	1	0	1	1	0	0	1	0	0	1	1	0	1	x	<table><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	1	0	0	0	1	1	1	0	1		<table><tr><td>5</td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>	5																								
1	0	0	1	0																																																											
0	1	1	0	1																																																											
1	0	1	0	1																																																											
1	0	0	1	0																																																											
0	1	1	0	1																																																											
1	0	0																																																													
0	1	1																																																													
1	0	1																																																													
5																																																															
→																																																															
<table><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr></table>	1	0	0	1	0	0	1	1	0	1	1	0	1	0	1	1	0	0	1	0	0	1	1	0	1	x	<table><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	1	0	0	0	1	1	1	0	1		<table><tr><td>5</td><td>1</td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>	5	1																							
1	0	0	1	0																																																											
0	1	1	0	1																																																											
1	0	1	0	1																																																											
1	0	0	1	0																																																											
0	1	1	0	1																																																											
1	0	0																																																													
0	1	1																																																													
1	0	1																																																													
5	1																																																														
→																																																															
<table><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr></table>	1	0	0	1	0	0	1	1	0	1	1	0	1	0	1	1	0	0	1	0	0	1	1	0	1	x	<table><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	1	0	0	0	1	1	1	0	1		<table><tr><td>5</td><td>1</td><td>3</td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>	5	1	3																						
1	0	0	1	0																																																											
0	1	1	0	1																																																											
1	0	1	0	1																																																											
1	0	0	1	0																																																											
0	1	1	0	1																																																											
1	0	0																																																													
0	1	1																																																													
1	0	1																																																													
5	1	3																																																													

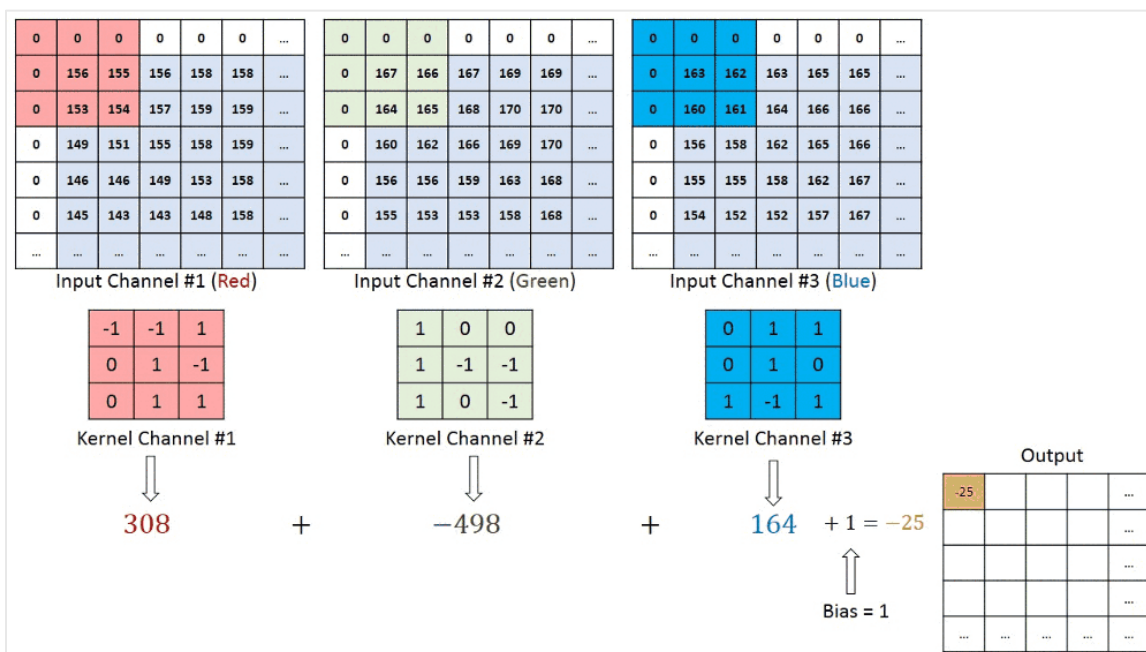
Tính toán tích chập lần lượt trên các vị trí

- Công thức tính toán cho các vị trí trên ma trận tích chập:



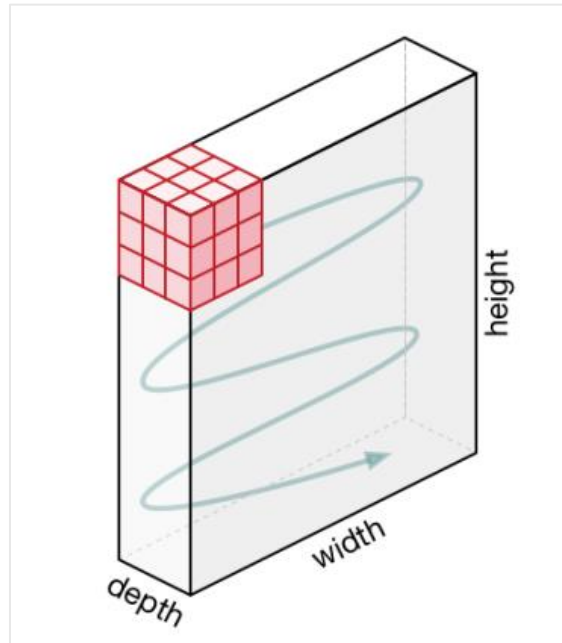
Công thức tính toán giá trị cho các vị trí tích chập

- Phép tích chập trên ảnh màu 3 kênh RGB:



Tích chập trên ảnh màu ba kênh

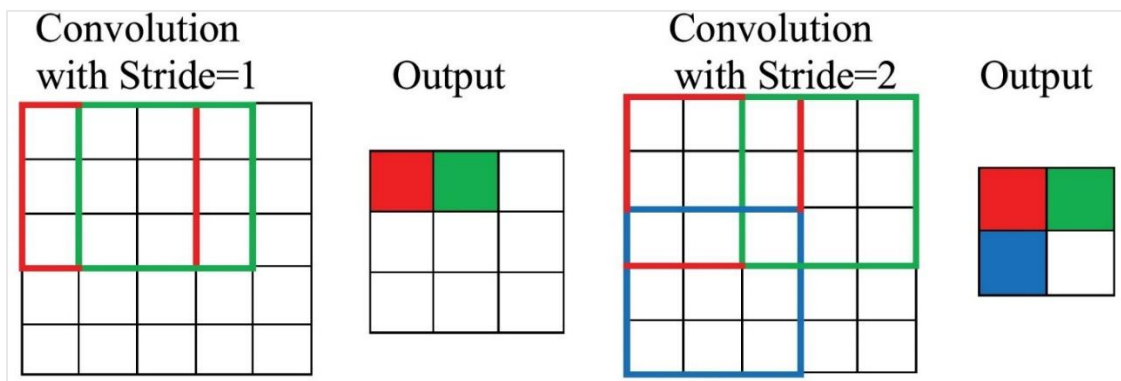
- Các kernel sẽ trượt trên toàn bộ bức hình và các feature map:



Thứ tự thực hiện tích chập của các kernel

Stride (bước nhảy)

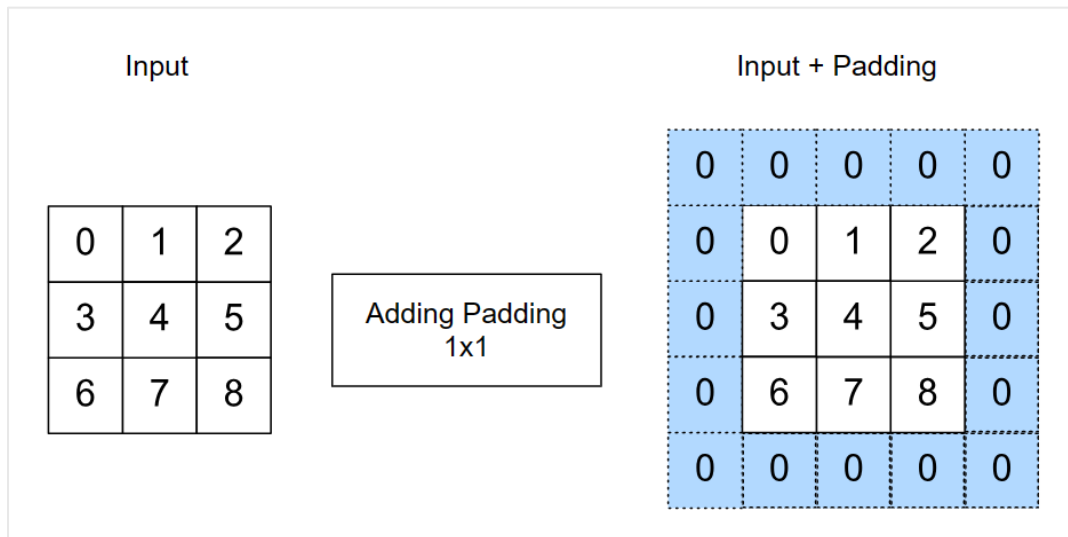
- Là số bước kernel di chuyển mỗi lần.
- Stride nhỏ $\rightarrow$ giữ nhiều thông tin hơn
- Stride lớn $\rightarrow$ giảm kích thước nhanh hơn



Bước nhảy trong tích chập

Padding (viền)

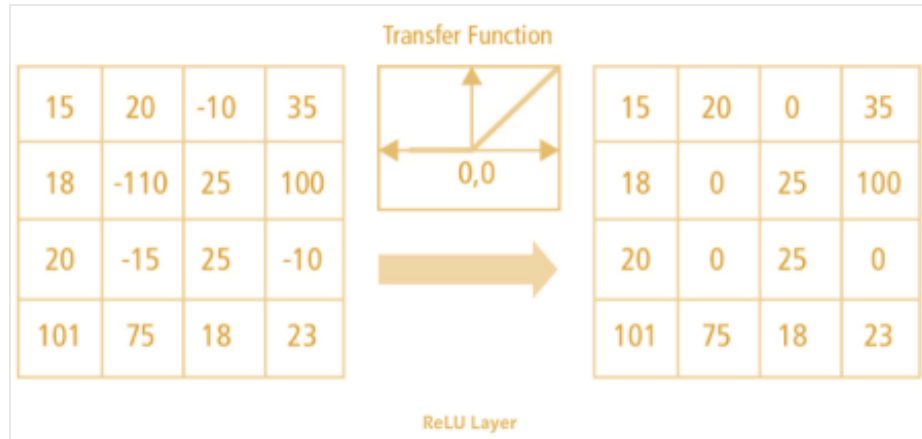
- Thêm giá trị (thường là 0) vào biên ảnh.
- Giúp:
 - Không mất thông tin biên
 - Giữ kích thước đầu ra



Viền trong tích chập

Lớp kích hoạt (Activation Layer)

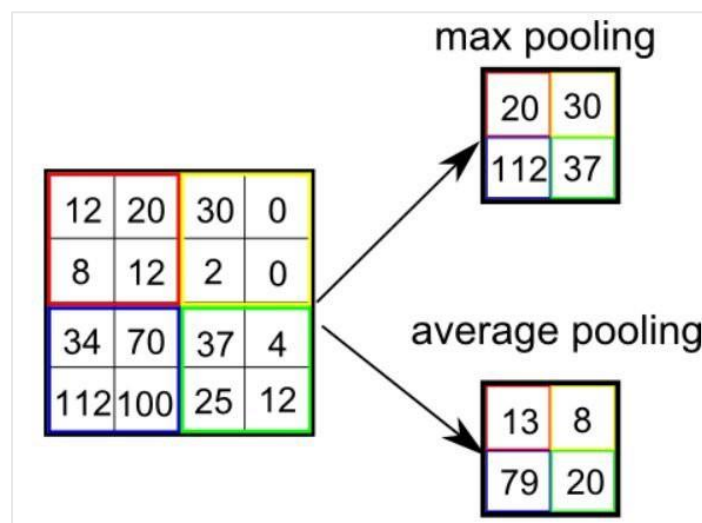
- Được áp dụng sau lớp tích chập.
- Tạo tính phi tuyến cho mô hình.
- Hàm phổ biến: **ReLU**
 - Giữ giá trị dương
 - Loại bỏ giá trị âm



Sử dụng hàm kích hoạt trong mạng tích chập

Lớp gộp (Pooling Layer)

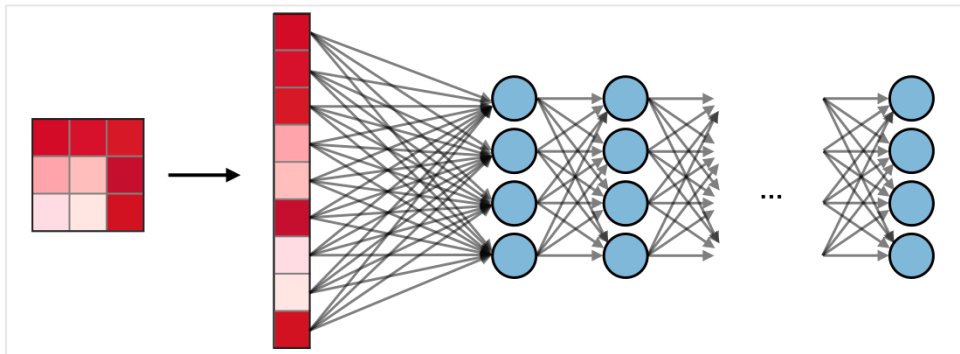
- Giảm kích thước feature map.
- Giảm số tham số và chi phí tính toán.
- Hai loại phổ biến:
 - **Max pooling:** lấy giá trị lớn nhất
 - **Average pooling:** lấy giá trị trung bình



Lớp gộp trong mạng tích chập

Lớp kết nối đầy đủ (Fully Connected Layer)

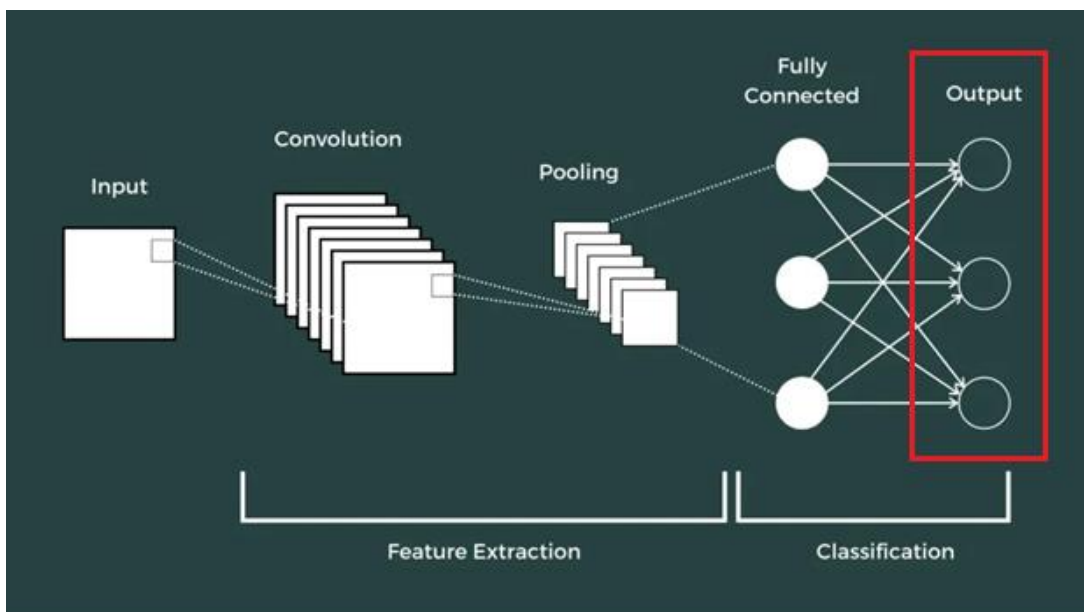
- Chuyển đặc trưng thành vector.
- Học mối quan hệ giữa các đặc trưng.
- Dùng để thực hiện phân loại.



Lớp kết nối đầy đủ trong mạng tích chập

Lớp đầu ra (Output Layer)

- Trả về kết quả dự đoán cuối cùng.
- Số neuron phụ thuộc số lớp cần phân loại.



Lớp đầu ra của mạng tích chập

Độ phức tạp phát hiện hình ảnh tăng dần: Quá trình trích xuất các đặc trưng của một bức ảnh thông qua mạng tích chập có thể được chia thành các mức độ khác nhau. Ở các đặc trưng bậc thấp (low-level features) các đặc trưng thu được thường là các đặc trưng như các cạnh ngang, cạnh dọc, cạnh chéo, các đường cong hay các dạng cơ bản. Ở các lớp tiếp theo mid-level features các đặc trưng ở mức tổng quát hơn càng về sau mạng càng có khả năng phát hiện các đường nét phức tạp đã rõ hình thù và đã có thể cấu thành vật thể chẳng hạn khi các đặc trưng ở các cấp trước sau khi tổng hợp lại có thể cấu thành nên các bộ phận lớn hơn như mắt, mũi, miệng,... còn ở các lớp cuối cùng chính là đặc trưng của đối tượng được tổng hợp được khi kết hợp các bộ phận trước đó và từ đó quá trình phân lớp được diễn ra cho đối tượng.



Các đặc trưng của mạng tích chập ở các độ sâu khác nhau của mạng

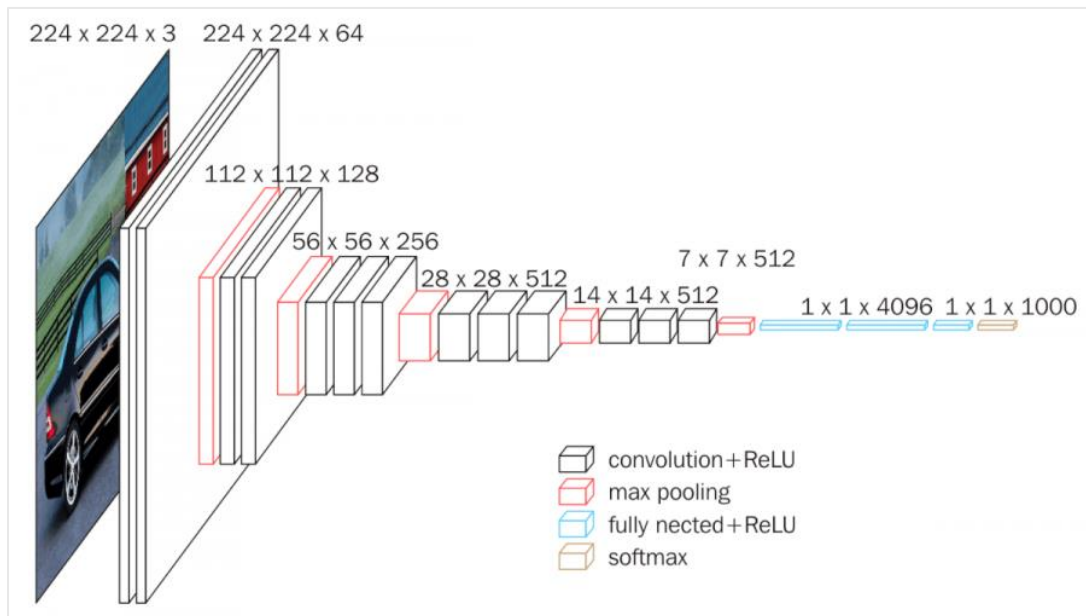
Một số các mạng tích chập

Mạng VGG16

Mạng VGG16 là một kiến trúc mạng nơ-ron tích chập (CNN) được giới thiệu bởi nhóm nghiên cứu Visual Geometry Group (VGG) thuộc Đại học Oxford là một trong những mô hình CNN nổi tiếng và phổ biến được sử dụng rộng rãi trong lĩnh vực thị giác máy tính và học sâu.

Kiến trúc VGG16 có tổng cộng 16 lớp (do đó tên gọi VGG16), trong đó bao gồm 13 lớp tích chập và 3 lớp kết nối đầy đủ (fully connected layers). Các lớp tích chập sử dụng các bộ lọc nhỏ kích thước 3x3 với các bước nhảy (stride) bằng 1 và đệm (padding) để duy trì kích thước đầu vào. VGG16 có đặc điểm là có số lượng lớp rất lớn và các kênh đầu ra rất

sâu, tạo ra một mô hình rất mạnh nhưng cũng rất phức tạp. Nó thường được sử dụng để thực hiện các tác vụ như phân loại ảnh, phát hiện đối tượng và trích xuất đặc trưng.



Kiến trúc mạng VGG16

Kiến trúc của VGG16 bao gồm:

- Lớp đầu vào: Một bức ảnh với kích thước cố định (thường là 224x224x3) được đưa vào mô hình.
- Lớp tích chập (Convolutional layers): VGG16 sử dụng một chuỗi các lớp tích chập với bộ lọc kích thước 3x3, hàm kích hoạt ReLU và padding để duy trì kích thước đầu vào. Các lớp tích chập này được xếp chồng lên nhau và sau đó được theo sau bởi các lớp max pooling để giảm kích thước.

VGG16 sử dụng một số lớp tích chập để trích xuất các đặc trưng của ảnh. Các lớp tích chập được thiết kế để nhận diện các đặc trưng cấp thấp (ví dụ: cạnh, góc) và các đặc trưng cấp cao (ví dụ: hình dạng, texture). VGG16 đã đạt được kết quả tốt trong nhiều nhiệm vụ phân loại ảnh và đã trở thành một trong những kiến trúc CNN quan trọng và phổ biến.

Bên cạnh VGG16 chúng ta còn có các mạng VGG khác như VGG11, VGG13, VGG19 với số lượng các lớp và kiến trúc các khối khác nhau.

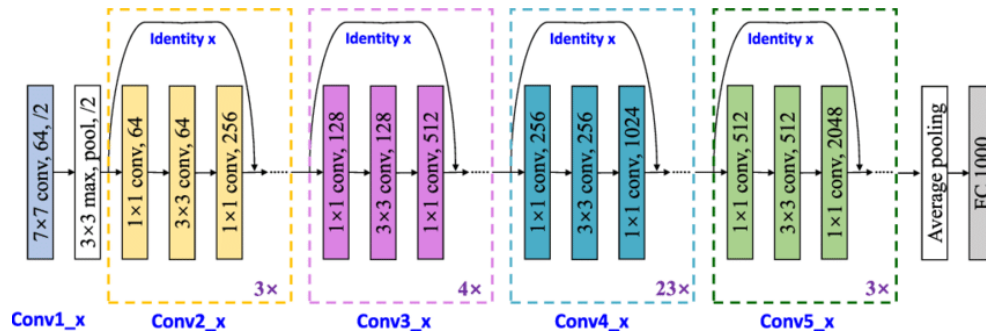
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Một số kiến trúc khác của mạng VGG

Mạng ResNet

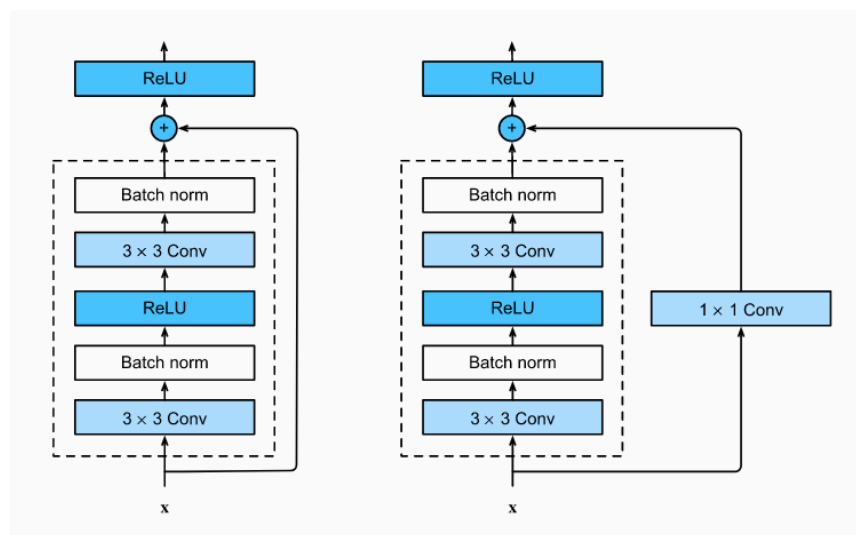
ResNet (Residual Network) là một kiến trúc mạng nơ-ron tích chập (CNN) đặc biệt được đề xuất bởi Kaiming He và các đồng nghiệp vào năm 2015. Nó đã đạt được thành công lớn trong các nhiệm vụ phân loại ảnh và xếp hạng trong cuộc thi ImageNet 2015.

Một trong những đặc điểm độc đáo của ResNet là việc sử dụng các khối cơ bản được gọi là Residual Blocks. Những khối này cho phép mạng xây dựng các đường truyền thông qua các lớp mà không bị giảm đáng kể hiệu suất, thậm chí có thể cải thiện nó. Kiến trúc ResNet sử dụng một số khối cơ bản để tạo nên mô hình. Dưới đây là một mô tả tổng quan về kiến trúc ResNet:



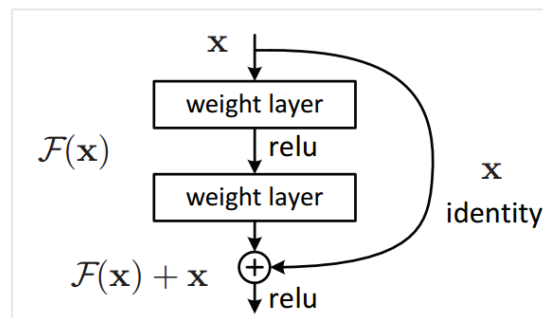
Kiến trúc mạng ResNet

Residual Block: Đây là thành phần chính của ResNet. Mỗi khối cơ bản bao gồm hai hoặc ba lớp tích chập kết hợp với các kết nối nhánh (skip connections) để truyền thông qua các lớp. Kết nối nhánh cho phép thông tin trực tiếp từ các lớp trước đó được truyền qua các lớp tiếp theo. Điều này giúp tránh hiện tượng mất thông tin (vanishing gradient). Trong mạng ResNet, kết nối tắt được áp dụng thông qua các khối phần dư (Residual Blocks). Mỗi khối phần dư bao gồm một kết nối tắt, còn được gọi là kết nối nhánh (shortcut connection), nối trực tiếp từ đầu vào đến đầu ra của khối. Cụ thể, khi thông tin đi qua một khối phần dư, thông tin từ đầu vào được truyền thẳng tới đầu ra thông qua kết nối tắt. Ngoài ra, để đảm bảo rằng thông tin từ đầu vào có thể được kết hợp với đầu ra của khối phần dư, đôi khi có thể sử dụng một lớp tích chập 1x1 để điều chỉnh kích thước của đầu vào để phù hợp với đầu ra.



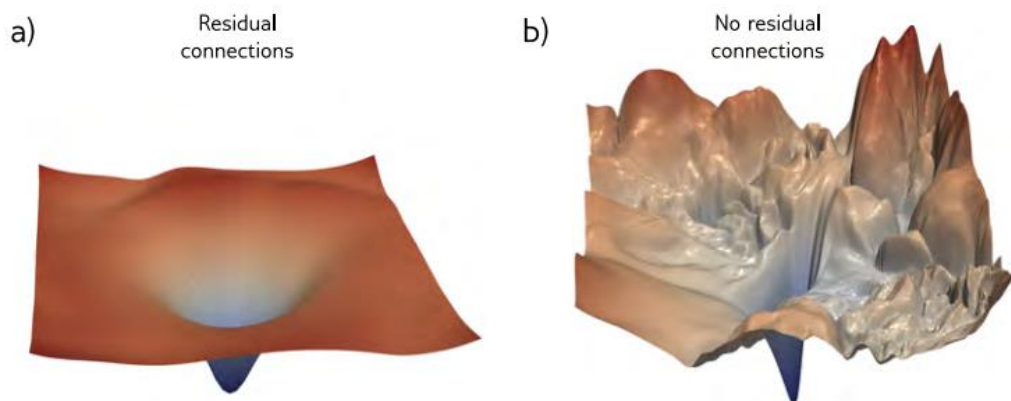
Kiến trúc của một khối phần dư

Mạng trích xuất đặc trưng: ResNet sử dụng một chuỗi các khối cơ bản để trích xuất các đặc trưng từ ảnh. Số lượng và kích thước của các khối cơ bản phụ thuộc vào phiên bản cụ thể của ResNet (ví dụ: ResNet-50, ResNet-101, ResNet-152). Các khối cơ bản được xếp chồng lên nhau để xây dựng một mạng CNN sâu. Những kiến trúc trước đây thường cải tiến độ chính xác nhờ gia tăng chiều sâu của mạng CNN. Nhưng thực nghiệm cho thấy đến một ngưỡng độ sâu nào đó thì độ chính xác của mô hình sẽ bão hòa và thậm chí phản tác dụng và làm cho mô hình kém chính xác hơn. Khi đi qua quá nhiều tầng độ sâu có thể làm thông tin gốc bị mất đi thì các nhà nghiên cứu của Microsoft đã giải quyết vấn đề này trên ResNet bằng cách sử dụng các kết nối tắt này.



Kết nối tắt trong mạng ResNet

Khi có áp dụng Residual connections khả năng học tập thông qua các mạng có độ sâu lớn được cải thiện, trong quá trình học mạng sẽ dễ giữ lại các đặc trưng đã học ở trước đó và kết quả cho ra của mô hình sẽ tốt hơn.



So sánh hình dạng hàm mất mát khi có kết nối phần dư và không có kết nối phần dư

ResNet là một kiến trúc mạng nổi tiếng và quan trọng trong lĩnh vực phân loại ảnh. Nó đã giúp nâng cao hiệu suất và khả năng học của các mô hình CNN, đồng thời vượt qua các thách thức của việc xây dựng các mạng sâu.

III. Bài tập

Bài tập 1. Thuật toán Gradient Descent:

Thêm thư viện:

```
import numpy as np
import matplotlib.pyplot as plt
```

Hàm *gradient_descent* nhận vào hàm số, đạo hàm của hàm số, giá trị khởi tạo, learning rate và step:

```
def gradient_descent(f, df, x_init, lr=0.1, steps=10):
    x_vals = [x_init]
    x = x_init
    print(f"{'Step':<5} {'x':>10} {'f(x)':>15}")
    print("-" * 30)
    for i in range(steps):
        fx = f(x)
        print(f"{i:<5} {x:>10.5f} {fx:>15.5f}")
        x = x - lr * df(x)
        x_vals.append(x)
    fx = f(x)
    print(f"{steps:<5} {x:>10.5f} {fx:>15.5f}")
    return x_vals
```

Hàm vẽ biểu đồ minh họa kết quả:

```
def plot_descent(f, df, x_init, lr, steps, title, xlim=(-10, 10), ylim=None):
    x_vals = gradient_descent(f, df, x_init, lr, steps)
    x_range = np.linspace(xlim[0], xlim[1], 1000)
    y_range = f(x_range)

    plt.figure(figsize=(10, 6))
    plt.plot(x_range, y_range, label='Function', color='blue')
    plt.scatter(x_vals, [f(x) for x in x_vals], color='red', label='Descent Steps')

    for i in range(len(x_vals)-1):
        plt.arrow(x_vals[i], f(x_vals[i]),
                  x_vals[i+1] - x_vals[i],
                  f(x_vals[i+1]) - f(x_vals[i]),
                  head_width=0.3, color='green', length_includes_head=True)

    plt.title(title)
    plt.xlabel('x')
    plt.ylabel('f(x)')
    plt.grid(True)
    plt.axhline(0, color='black', linewidth=0.5)
    plt.axvline(0, color='black', linewidth=0.5)
    plt.legend()
    plt.xlim(xlim)
    if ylim:
        plt.ylim(ylim)
    plt.show()
```

Thử nghiệm với hàm bậc 2, bậc 3, bậc 4:

```
f1 = lambda x: x**2
df1 = lambda x: 2*x

f2 = lambda x: x**3
df2 = lambda x: 3*x**2

f3 = lambda x: x**4 - 3*x**3 + 2
df3 = lambda x: 4*x**3 - 9*x**2
```

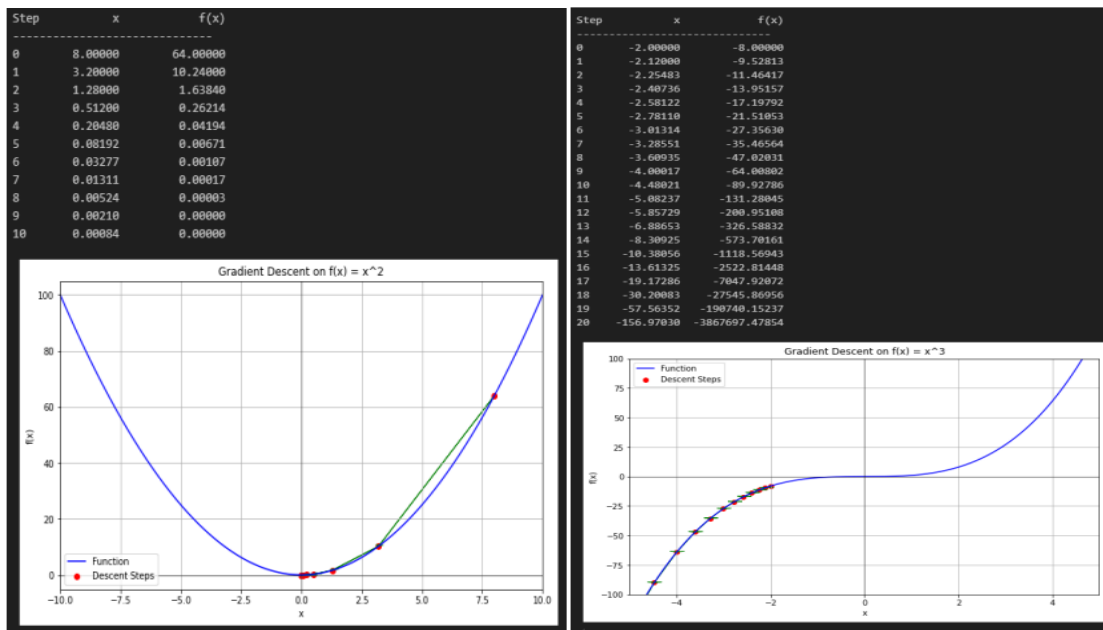
Gọi hàm vẽ biểu đồ để quan sát kết quả:

```
plot_descent(f1, df1, x_init=8, lr=0.3, steps=10, title='Gradient Descent on f(x) = x^2', xlim=(-10, 10))

plot_descent(f2, df2, x_init=-2, lr=0.01, steps=20,
             title='Gradient Descent on f(x) = x^3',
             xlim=(-5, 5), ylim=(-100, 100))

plot_descent(f3, df3, x_init=6, lr=0.01, steps=30, title='Gradient Descent on f(x) = x^4 - 3x^3 + 2', xlim=(-2, 6), ylim=(-10, 20))
```


Kết quả sẽ trả về giá trị của x , giá trị của hàm $f(x)$ cùng với biểu đồ quá trình thay đổi:



Thử nghiệm với bài toán hồi quy

Thêm thư viện cần thiết:

```
import numpy as np
import matplotlib.pyplot as plt
```

Tạo dữ liệu giả định (giá trị w cần tìm thực sự là 3):

```
np.random.seed(0)
x_data = np.linspace(0, 10, 20)
y_data = 3.0 * x_data + np.random.randn(x_data.shape) * 3 # true w = 3.0
```

Thiết lập khởi tạo của trọng số, learning rate và step:

```
w = 0.0
lr = 0.001
steps = 50
```

Tạo biến để lưu lại giá trị trọng số và loss sau mỗi lần thay đổi:

```
w_history = []
loss_history = []
```

Biểu đồ trực quan kết quả thực hiện:

```
plt.figure(figsize=(10, 6))
plt.scatter(x_data, y_data, label='Data', color='blue')
```

Tiến hành khởi chạy thuật toán:

```
print(f"{'Step':<5} {'w':>8} {'Loss':>12} {'grad':>10}")
print("-" * 35)

for step in range(steps):
    y_pred = w * x_data
    loss = np.mean((y_pred - y_data) ** 2)
    grad = np.mean(2 * x_data * (y_pred - y_data))

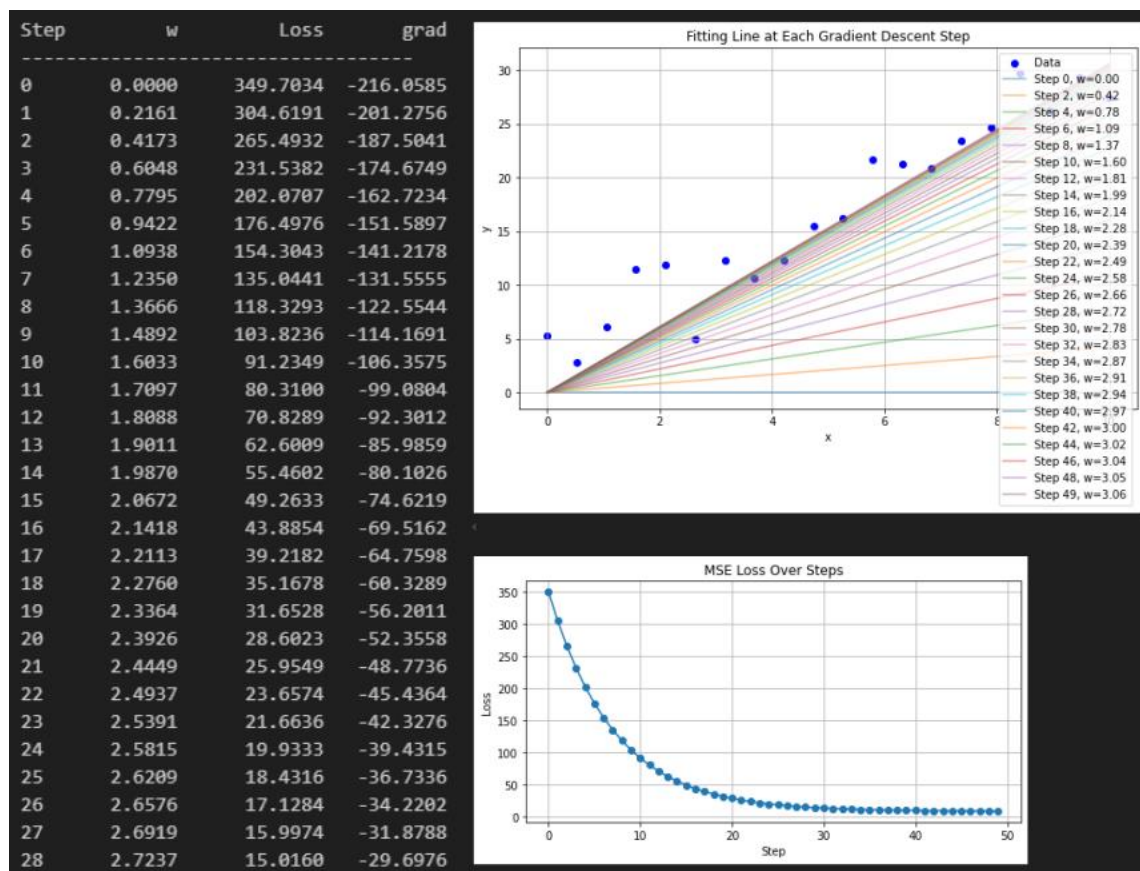
    print(f"{'step':<5} {'w':>8.4f} {'loss':>12.4f} {'grad':>10.4f}")

    w_history.append(w)
    loss_history.append(loss)

    # Plot fit line for this step
    if step % 2 == 0 or step == steps - 1: # show every few steps
        plt.plot(x_data, w * x_data, label=f'Step {step}, w={w:.2f}', alpha=0.5)

    # Update weight
    w -= lr * grad
```

Kết quả minh họa của hàm số tìm được với mỗi w khác nhau và độ lỗi tương ứng:



Bài tập 2. Trong bài tập này, sinh viên sẽ xây dựng hệ thống **phân loại ảnh chữ số viết tay (0–9)** sử dụng bộ dữ liệu **MNIST**.

Mỗi ảnh đầu vào có kích thước **28×28 pixel (grayscale)**, nhiệm vụ của mô hình là dự đoán ảnh thuộc chữ số nào từ 0 đến 9.

Bài toán được thực hiện theo hai hướng:

- **Neural Network (MLP):** mạng fully-connected truyền thống
- **Convolutional Neural Network (CNN):** mạng tích chập cho ảnh

Phần 1: Neural Network (MLP)

- Chuyển ảnh từ ma trận 28×28 thành vector 784 chiều
- Xây dựng mạng gồm các lớp:
 - Input → Hidden → Hidden → Output
- Sử dụng hàm kích hoạt ReLU
- Huấn luyện và đánh giá độ chính xác

Phần 2: Convolutional Neural Network (CNN)

- Giữ nguyên cấu trúc ảnh 2D
- Sử dụng các lớp:
 - Convolution
 - ReLU
 - MaxPooling
- Sau đó đưa qua fully-connected để phân loại

Phần 3: So sánh

- So sánh kết quả giữa MLP và CNN:

- Accuracy
- Loss
- Nhận xét về hiệu quả của từng mô hình

Hướng dẫn code

Import thư viện

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

import matplotlib.pyplot as plt
```

Khởi tạo các siêu tham số, kiểm tra GPU:

```
BATCH_SIZE = 64
EPOCHS = 5
LR = 0.001

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

Khởi tạo dataset, tạo tập train và tập test:

```
transform = transforms.ToTensor()

train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    transform=transform,
    download=True
)

test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    transform=transform,
    download=True
)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE)
```

Gọi một đối tượng ảnh trong tập dataset:

```
images, labels = next(iter(train_loader))

plt.imshow(images[0].squeeze(), cmap='gray')
plt.title(f"Label: {labels[0].item()}")
plt.show()
```

Kiến trúc mạng MLP sử dụng:

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()

        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.net(x)
```

Kiến trúc mạng CNN sử dụng:

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*7*7, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.fc(x)
        return x
```

Hàm huấn luyện, hàm này đưa dữ liệu vào mô hình sau đó tính toán và cập nhật trọng số:

```
def train(model, loader, optimizer, criterion):
    model.train()
    total_loss = 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    return total_loss / len(loader)
```

Hàm đánh giá:

```
def evaluate(model, loader):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    return correct / total
```

Tiến hành huấn luyện mô hình:

```
model = MLP().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

print("Training MLP...")

for epoch in range(EPOCHS):
    loss = train(model, train_loader, optimizer, criterion)
    acc = evaluate(model, test_loader)

    print(f"Epoch {epoch+1}: Loss={loss:.4f}, Acc={acc:.4f}")
```


Độ lỗi và độ chính xác sau mỗi Epoch của mô hình MLP:

```
Training MLP...
Epoch 1: Loss=0.3389, Acc=0.9506
Epoch 2: Loss=0.1374, Acc=0.9617
Epoch 3: Loss=0.0942, Acc=0.9688
Epoch 4: Loss=0.0715, Acc=0.9732
Epoch 5: Loss=0.0567, Acc=0.9723
```

Tương tự với mô hình CNN:

```
model = CNN().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

print("Training CNN...")

for epoch in range(EPOCHS):
    loss = train(model, train_loader, optimizer, criterion)
    acc = evaluate(model, test_loader)

    print(f"Epoch {epoch+1}: Loss={loss:.4f}, Acc={acc:.4f}")
```

✓ 1m 34.9s

```
Training CNN...
Epoch 1: Loss=0.1864, Acc=0.9807
Epoch 2: Loss=0.0529, Acc=0.9884
Epoch 3: Loss=0.0355, Acc=0.9898
Epoch 4: Loss=0.0270, Acc=0.9866
Epoch 5: Loss=0.0195, Acc=0.9891
```

Bài tập 3. Trong bài tập này, sinh viên xây dựng hệ thống **phân loại ảnh màu CIFAR-10**

Bộ dữ liệu **CIFAR-10** gồm 10 lớp ảnh:

- airplane
- automobile
- bird
- cat
- deer
- dog
- frog
- horse
- ship
- truck

Mỗi ảnh có kích thước $32 \times 32 \times 3$.

Bài tập gồm 3 mô hình:

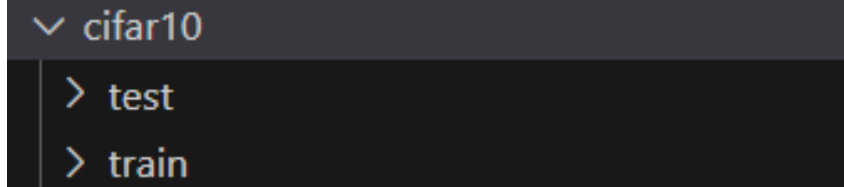
1. **CNN cơ bản**
2. **VGG**
3. **ResNet**

Mục tiêu là huấn luyện, đánh giá và so sánh hiệu quả giữa các kiến trúc.

Link dataset: <https://www.kaggle.com/datasets/ayush1220/cifar10>

(Nên sử dụng GPU, có thể code trực tiếp trên Kaggle)

Cấu trúc thư mục huấn luyện gồm tập train và test:



```
▼ cifar10
  > test
  > train
```

Import các thư viện cần thiết:

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

Khởi tạo giá trị siêu tham số, kiểm tra GPU:

```
BATCH_SIZE = 128
EPOCHS = 10
LR = 0.001

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

Khởi tạo bộ dữ liệu train và test:

```
from torchvision.datasets import ImageFolder

TRAIN_DIR = "cifar10/train"
TEST_DIR = "cifar10/test"

transform_train = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                        (0.5, 0.5, 0.5))
])

transform_test = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                        (0.5, 0.5, 0.5))
])

train_dataset = ImageFolder(root=TRAIN_DIR, transform=transform_train)
test_dataset = ImageFolder(root=TEST_DIR, transform=transform_test)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

classes = train_dataset.classes
print("Classes:", classes)
```

Kiểm tra một phần tử trong bộ dữ liệu huấn luyện:

```
images, labels = next(iter(train_loader))

img = images[0] / 2 + 0.5  # unnormalize
img = img.permute(1, 2, 0)

plt.imshow(img)
plt.title(f"Label: {classes[labels[0].item()]}")
plt.axis("off")
plt.show()

✓ 0.1s
```



Hàm huấn luyện mô hình:

```
from tqdm.notebook import tqdm, trange

def train(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    pbar = tqdm(loader, desc="Train", leave=False)

    for images, labels in pbar:
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        pbar.set_postfix({
            "loss": f"{loss.item():.4f}",
            "acc": f"{correct / total:.4f}"
        })

    avg_loss = total_loss / len(loader)
    avg_acc = correct / total
    return avg_loss, avg_acc
```

Hàm đánh giá mô hình:

```
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    pbar = tqdm(loader, desc="Eval", leave=False)

    with torch.no_grad():
        for images, labels in pbar:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item()

            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

            pbar.set_postfix({
                "loss": f"{loss.item():.4f}",
                "acc": f"{correct / total:.4f}"
            })

    avg_loss = total_loss / len(loader)
    avg_acc = correct / total
    return avg_loss, avg_acc
```

Hàm triển khai quá trình huấn luyện:

```
def run_training(model, train_loader, test_loader, criterion, optimizer, device, epochs, model_name="Model"):
    train_losses, train_accs = [], []
    test_losses, test_accs = [], []

    print(f"Training {model_name}...")

    for epoch in trange(epochs, desc=f"{model_name} Epoch"):
        train_loss, train_acc = train(model, train_loader, optimizer, criterion, device)
        test_loss, test_acc = evaluate(model, test_loader, criterion, device)

        train_losses.append(train_loss)
        train_accs.append(train_acc)
        test_losses.append(test_loss)
        test_accs.append(test_acc)

        print(
            f"Epoch {epoch+1}/{epochs} | "
            f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
            f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.4f}"
        )

    return train_losses, train_accs, test_losses, test_accs
```

Khởi tạo kiến trúc CNN huấn luyện:

```
class BasicCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), # 32x32
            nn.ReLU(),
            nn.MaxPool2d(2), # 16x16

            nn.Conv2d(32, 64, kernel_size=3, padding=1), # 16x16
            nn.ReLU(),
            nn.MaxPool2d(2), # 8x8

            nn.Conv2d(64, 128, kernel_size=3, padding=1), # 8x8
            nn.ReLU(),
            nn.MaxPool2d(2) # 4x4
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

Huấn luyện mô hình CNN vừa khởi tạo:

```
model = BasicCNN(num_classes=10).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

cnn_train_losses, cnn_train_accs, cnn_test_losses, cnn_test_accs = run_training(
    model=model,
    train_loader=train_loader,
    test_loader=test_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    epochs=EPOCHS,
    model_name="BasicCNN"
)
```

Khởi tạo mô hình VGG16 và huấn luyện:

```
model = models.vgg16(weights=models.VGG16_Weights.DEFAULT)
model.classifier[6] = nn.Linear(model.classifier[6].in_features, 10)
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

vgg_train_losses, vgg_train_accs, vgg_test_losses, vgg_test_accs = run_training(
    model=model,
    train_loader=train_loader,
    test_loader=test_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    epochs=EPOCHS,
    model_name="VGG16"
)
```

Khởi tạo mô hình ResNet18 và huấn luyện:

```
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
model.fc = nn.Linear(model.fc.in_features, 10)
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

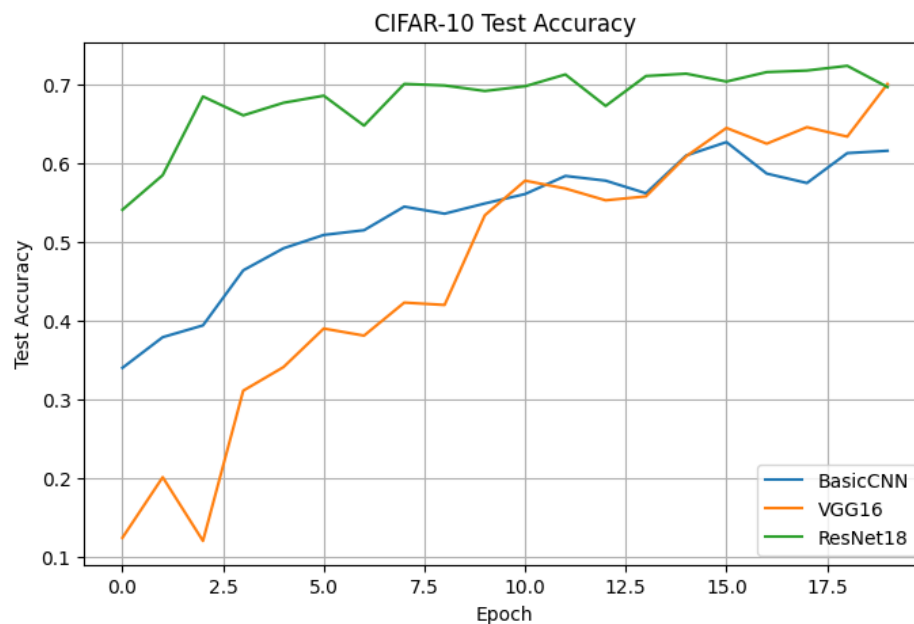
resnet_train_losses, resnet_train_accs, resnet_test_losses, resnet_test_accs = run_training(
    model=model,
    train_loader=train_loader,
    test_loader=test_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    epochs=EPOCHS,
    model_name="ResNet18"
)
```


Hàm kiểm tra độ chính xác của các mô hình sau mỗi epoch:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.plot(cnn_test_accs, label="BasicCNN")
plt.plot(vgg_test_accs, label="VGG16")
plt.plot(resnet_test_accs, label="ResNet18")
plt.xlabel("Epoch")
plt.ylabel("Test Accuracy")
plt.title("CIFAR-10 Test Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

Kết quả huấn luyện minh họa:



Bài tập 3. Trong bài tập này, sinh viên xây dựng hệ thống **phân loại ảnh màu** trên bộ dữ liệu **ImageNet**

Yêu cầu thực hiện với ba mô hình:

CNN cơ bản

VGG

ResNet

Mục tiêu là huấn luyện, đánh giá và so sánh hiệu quả giữa các kiến trúc.